



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

CSA:3712

Software Testing

CO1-AT1-DESCRIPTIVE WRITTEN EXAM

Name: B S Reshma

Reg no. 192521342

1. A banking application team is facing frequent failures due to unstructured testing practices. As a test engineer, apply the concept of testing as an engineering activity to redesign the testing approach. Explain how systematic test design, tools, and standards can be introduced. Illustrate how this approach improves defect detection and quality. Suggest suitable techniques to replace ad-hoc testing. Justify your solution with a practical example.
2. An atm system is producing inconsistent results due to lack of a defined testing process. As a tester, demonstrate how you would implement a structured testing process across sdlc phases. Explain each stage and its relevance to quality improvement. Apply your approach to atm withdrawal functionality. Discuss how process consistency reduces defects. Conclude with expected outcomes.
3. A web application shows mismatch between expected and actual outputs during login and transaction operations. Apply basic testing definitions to identify errors, defects, and failures in this scenario. Illustrate their relationships with suitable examples. Explain how improper understanding of these terms affects testing. Suggest how to improve defect reporting clarity. Justify your answer with examples.
4. A software project team observes that most defects are concentrated in a few modules, leading to delays. Apply software testing principles to address this issue. Explain how defect clustering and other principles guide test planning. Illustrate how test cases can be prioritized effectively. Discuss how applying these principles improves testing efficiency. Provide a real-time example.
5. A mobile banking application frequently crashes during fund transfer operations. As a tester, apply software testing principles to identify the root cause of failures. Explain how systematic testing can reduce such issues. Illustrate your approach with suitable examples. Suggest improvements in testing practices to avoid future failures.
6. A company maintains a defect repository but fails to utilize it effectively, resulting in repeated defects across releases. As a tester, apply defect classification and repository concepts to improve the situation. Explain how defects originate and how they can be categorized. Illustrate how repository data can support test design and prevention. Suggest improvements to avoid recurring defects. Conclude with expected quality improvements.

7. A development team confuses debugging with testing, leading to ineffective defect resolution. Apply the concepts of testing and debugging to differentiate both processes. Explain their roles in software quality. Illustrate how improper usage affects defect detection. Suggest a structured approach to integrate both effectively.
8. An e-commerce platform shows inconsistent behavior across browsers. Apply testing principles to design a strategy for cross-browser testing. Explain how defects originate in such scenarios. Illustrate how systematic testing improves reliability. Suggest tools and techniques for better coverage.
9. A payroll system produces incorrect salary calculations due to logical errors in code. Apply defect classification techniques to categorize these defects. Explain how understanding defect classes improves testing. Illustrate how such defects can be prevented in future releases. Suggest improvements in testing strategy.
10. A software team skips proper test case design and directly executes random tests. Apply testing as an engineering activity to redesign the testing process. Explain the importance of structured test case design. Illustrate how this improves defect detection. Suggest suitable techniques to enhance test effectiveness.
11. A healthcare application shows data inconsistency across modules. Apply testing principles to analyze the issue. Explain how defects may originate during development. Illustrate how systematic testing ensures data integrity. Suggest improvements in defect tracking and reporting.
12. A company faces repeated defects in its software releases despite testing efforts. Apply the concept of defect repository to analyze the issue. Explain how repository data supports test design. Illustrate how defect patterns can be identified. Suggest improvements to prevent recurring defects.
13. A web-based learning system fails during peak usage due to untested scenarios. Apply software testing principles to identify gaps in testing. Explain how test planning and coverage impact performance. Illustrate how better test strategies improve system reliability. Suggest enhancements to avoid such failures.
14. A team observes that testers are not involved in early development stages. Apply the role of tester in sdhc to improve this situation. Explain how early involvement impacts quality. Illustrate benefits using a practical example. Suggest changes in team collaboration practices.

15. A system produces correct outputs in some cases but fails in others due to incomplete test coverage. Apply testing principles to identify gaps. Explain how proper coverage ensures reliability. Illustrate how to improve test case design. Suggest techniques for better validation.
16. A logistics application shows delay in defect identification due to poor reporting practices. Apply testing concepts to improve defect reporting. Explain how clear definitions of defects help in tracking. Illustrate how structured reports enhance communication. Suggest improvements in reporting standards.
17. A software team struggles to maintain consistency in testing across modules. Apply process-based testing approaches to solve the issue. Explain how standardized processes improve quality. Illustrate your approach with an example. Suggest methods to ensure consistency.
18. A system fails after deployment due to untested edge cases. Apply testing principles to identify why such cases were missed. Explain how boundary testing helps in such scenarios. Illustrate how to include edge cases in testing. Suggest improvements in test design.
19. A company lacks proper documentation of defects, making it difficult to track issues. Apply defect repository concepts to address this problem. Explain how documentation improves defect management. Illustrate how structured repositories support analysis. Suggest improvements in documentation practices.
20. A banking system shows repeated failures in transaction processing due to improper testing practices. Apply software testing principles to redesign the testing approach. Explain how structured testing improves quality. Illustrate your solution with examples. Suggest techniques to enhance defect detection.

ANSWERS:

1.

Aim:

To redesign the testing approach of a banking application by applying testing as an engineering activity using systematic test design, appropriate testing techniques, tools and standards, thereby improving defect detection, reliability and software quality.

Introduction:

Software testing as an engineering activity means that testing is not performed randomly or only when the software is completed. It is a planned, systematic, measurable and repeatable process that is integrated throughout the software development life cycle.

In a banking application, software failures can result in incorrect account balances, failed transactions, duplicate payments or incorrect fund transfers. Therefore, testing must be carefully designed according to requirements and risks.

The first step is to understand the requirements and identify what the system is expected to do. These requirements are then converted into test conditions and test cases. Test design techniques such as equivalence partitioning, boundary value analysis, decision table testing and state transition testing can be used instead of random testing.

Testing tools can also be used to manage test cases, execute automated tests and maintain defect reports. A standardized testing process ensures that all testers follow similar procedures and that defects can be traced from identification to resolution.

Procedure / testing process:

1. Requirement analysis

The tester first studies requirements such as login, account validation, balance checking, withdrawal, fund transfer and transaction limits.

For example, if a requirement states that a customer can transfer a maximum of ₹50,000 per transaction, the tester must identify valid and invalid conditions around this limit.

2. Test planning

A test plan is prepared by defining:

- Testing scope
- Testing objectives
- Resources
- Test environment
- Schedule

- Risks
- Testing techniques
- Entry and exit criteria

High-risk banking operations should receive greater testing attention.

3. Systematic test design

Instead of randomly selecting inputs, test cases are derived from requirements.

Equivalence partitioning divides inputs into valid and invalid groups.

Boundary value analysis tests values around limits.

Decision table testing tests combinations of conditions.

State transition testing verifies changes such as:

Transaction initiated → processing → successful / failed / pending

4. Test execution

Test cases are executed and actual results are compared with expected results.

Both positive and negative scenarios should be tested.

5. Defect reporting

If the actual result differs from the expected result, a defect is recorded with:

- Defect id
- Description
- Steps to reproduce
- Expected result
- Actual result
- Severity
- Priority
- Evidence

6. Retesting and regression testing

After the developer fixes a defect, the same test is executed again. This is retesting.

Other related functions are also tested to ensure that the modification has not introduced new defects. This is regression testing.

7. Automation

Repeated test cases, especially regression tests, can be automated. Automation reduces manual effort and allows the same tests to be executed consistently.

Practical application / example:

Consider a banking withdrawal system with a maximum withdrawal limit of ₹50,000.

A random tester may test only ₹5,000.

An engineering-based tester would test:

- ₹0
- ₹1
- ₹49,999
- ₹50,000
- ₹50,001
- Insufficient balance
- Invalid account
- Network interruption

For example, if ₹50,000 is incorrectly rejected even though it is the permitted maximum, boundary value analysis will identify the defect.

Similarly, if the network disconnects after the account is debited, the tester should verify that the transaction does not result in an incorrect or duplicate deduction.

Conclusion:

Testing as an engineering activity replaces ad-hoc testing with a systematic, structured and measurable process. The use of test design techniques, defect tracking, automation, regression testing and standardized procedures improves test coverage and defect detection. In a banking application, this approach reduces transaction failures and improves reliability, accuracy and overall software quality.

2.

Aim:

To implement a structured testing process throughout the software development life cycle for an atm withdrawal system and improve the reliability and consistency of atm transactions.

Introduction:

A structured testing process means that testing activities are performed systematically from the requirements stage until software deployment and maintenance. Testing is not treated as a final activity after coding.

For an atm system, important functions include card validation, pin verification, account validation, balance checking, withdrawal limits, cash dispensing and transaction recording.

Since atm transactions involve financial information, even a small defect can cause incorrect balance updates or failed transactions. Therefore, every sdlc phase should include appropriate testing activities.

Procedure / testing process

1. Requirement analysis

The tester examines requirements such as:

- Valid card verification
- Pin authentication
- Available balance
- Minimum withdrawal amount
- Maximum withdrawal limit
- Cash availability
- Transaction completion

Ambiguous requirements should be identified early.

2. Test planning

The tester defines the testing scope, resources, environment, test schedule, risks and acceptance criteria.

3. Test case design

Test cases are created for normal, invalid and boundary conditions.

For example:

- Correct card and pin
- Incorrect pin
- Expired card
- Insufficient balance
- Amount above withdrawal limit
- Atm without sufficient cash
- Network failure

4. Test environment setup

An atm simulation environment, test accounts, cards, database and required network configuration are prepared.

5. Test execution

All planned test cases are executed and actual results are compared with expected results.

6. Defect reporting

Any difference between expected and actual behavior is recorded as a defect with complete reproduction information.

7. Retesting

After the developer fixes a defect, the failed test case is executed again.

8. Regression testing

Other atm functions are tested to make sure the fix has not affected existing functionality.

9. Test closure

At the end, test results, defect status, coverage and unresolved risks are reviewed.

Practical application / example

Suppose an atm allows withdrawals between ₹100 and ₹50,000.

Scenario	Expected result
Valid card + correct pin + sufficient balance	Cash dispensed
Incorrect pin	Transaction rejected
Insufficient balance	Withdrawal rejected
₹50,001 withdrawal	Rejected
₹50,000 withdrawal	Accepted if balance and other conditions permit
Atm has insufficient cash	Appropriate message displayed
Network failure	Transaction safely handled

This structured process ensures that not only normal withdrawal is tested but also failure and edge conditions.

Conclusion:

A structured testing process across the sdlc ensures that defects are detected early and that testing is consistent and traceable. For atm withdrawal functionality, it reduces incorrect

deductions, failed transactions and authentication problems, resulting in a more reliable and dependable system.

3.

Aim:

To identify and differentiate errors, defects and failures in login and transaction operations and improve the clarity of defect reporting.

Introduction:

The terms error, defect and failure represent different stages of a software problem.

An error is a human mistake made during requirements, design, coding or another development activity.

A defect is a flaw introduced into the software as a result of an error.

A failure occurs when the software executes the defect and produces an incorrect or unexpected result.

The relationship can be represented as:

Human error → defect in software → execution → failure

Understanding this relationship is important because testers primarily observe failures and identify the corresponding defects, while developers generally investigate and correct the underlying cause.

Procedure/testing process:

1. Identify expected behaviour

The tester first determines what the application should do according to the requirements.

2. Execute the function

The tester performs login or transaction operations using different valid and invalid inputs.

3. Compare results

The actual output is compared with the expected output.

4. Identify failure

If the application behaves incorrectly, the observed behavior is identified as a failure.

5. Trace the defect

The development team investigates the software to identify the defect that caused the failure.

6. Identify the original error

The team may determine whether the defect originated from incorrect requirements, design or coding decisions.

7. Report the problem clearly

The defect report should clearly describe the observed behavior and provide steps for reproduction.

Practical application / example

Consider a banking login system.

The requirement states:

A user should be allowed to log in only when the correct username and password are entered.

Suppose the developer accidentally writes incorrect password validation logic.

The developer's incorrect coding decision is an error.

The incorrect authentication logic present in the program is a defect.

When a user enters an incorrect password but is still successfully logged in, the observed incorrect behavior is a failure.

Consider another example involving a transaction.

Suppose a customer transfers ₹5,000, but the application deducts ₹8,000.

The incorrect calculation decision is the error, the incorrect calculation logic is the defect, and the wrong deduction observed by the customer is the failure.

Conclusion:

Correctly understanding error, defect and failure improves communication between testers and developers. It allows defects to be reported clearly, traced to their causes and corrected efficiently. Clear terminology also prevents confusion during defect analysis and improves overall software quality.

4.

Aim:

To apply software testing principles, especially defect clustering, to prioritize testing activities and improve defect detection efficiency.

Introduction:

One important software testing principle is defect clustering. It states that a small number of modules often contain a large proportion of the defects.

This means that defects are not always evenly distributed throughout a system. Complex modules, frequently modified modules and modules involving complicated business logic may contain more defects.

Other testing principles also guide planning. Exhaustive testing is generally impossible, so testing effort must be prioritized according to risk, historical defects, complexity and business importance.

Procedure / testing process:

1. Collect historical defect information

Previous defect reports are analyzed to determine which modules have experienced the greatest number of defects.

2. Identify high-risk modules

Modules involving financial transactions, authentication, calculations or external integrations may require greater testing attention.

3. Analyze defect severity

Critical and high-severity defects should receive greater priority.

4. Prioritize test cases

Test cases associated with high-risk modules should be executed earlier.

5. Increase coverage

Additional positive, negative, boundary and integration tests can be designed for defect-prone modules.

6. Perform regression testing

When a defect-prone module is modified, regression tests should be executed to ensure that existing functionality remains correct.

7. Continuously update the plan

New defect information should be used to modify future testing priorities.

Practical application / example

Suppose an e-commerce application contains:

- Payment module – 40 defects
- Login module – 25 defects
- Product search – 5 defects
- Information page – 1 defect

The historical information indicates that payment and login are more defect-prone.

Therefore, testers can design more detailed test cases for payment and login, including invalid credentials, payment failures, boundary amounts, network interruptions and duplicate transactions.

This does not mean that the other modules are ignored. Rather, testing resources are distributed according to risk and available evidence.

Conclusion:

Defect clustering helps testers understand where defects are concentrated and use testing resources efficiently. Combining historical defect information with risk-based prioritization improves test effectiveness, coverage and early detection of important defects.

5.

Aim:

To improve defect management by classifying defects and effectively using a defect repository to identify patterns and prevent repeated defects.

Introduction

A defect repository is a centralized location where information about software defects is recorded and maintained.

Defects may originate during different stages of software development. Incorrect requirements may produce requirement defects, poor design may produce design defects, incorrect coding may produce implementation defects, and integration problems may produce interface or integration defects.

Defects can also be classified according to their nature, such as functional, performance, security, usability, data and interface defects.

Procedure / testing process:

1. Identify the defect

The tester identifies a difference between expected and actual behavior.

2. Record the defect

The defect is entered into the repository with a unique id.

3. Classify the defect

The defect is classified according to its type, module, severity and priority.

4. Identify root cause

The team investigates whether the problem originated from requirements, design, coding, data or integration.

5. Correct the defect

The developer implements the appropriate correction.

6. Retest

The tester verifies that the defect has been corrected.

7. Regression test

Related functionality is tested to make sure the correction has not created new problems.

8. Analyze historical data

The repository is periodically reviewed to identify repeated defect patterns.

Practical application / example:

Suppose a banking application repeatedly contains payment calculation defects.

The repository may show that most defects occur in the payment calculation module and are caused by incorrect rounding logic.

The testing team can respond by:

- Adding calculation test cases.
- Testing boundary values.
- Adding regression tests.
- Reviewing the calculation requirements.
- Performing code reviews.
- Automating important calculation tests.

The repository therefore becomes a source of information for future test design and prevention, rather than simply a list of old bugs.

Conclusion:

Effective defect classification and repository usage help organizations understand where defects originate, identify recurring problems and improve future testing. This reduces repeated defects and improves software quality across releases.

6.

Aim

To identify the root causes of fund-transfer failures in a mobile banking application and establish a systematic testing approach to prevent future crashes.

Introduction

A mobile banking application must correctly handle financial transactions under normal and abnormal conditions. A crash during fund transfer may occur because of incorrect business logic, invalid inputs, network interruptions, database problems, concurrency issues or improper error handling.

Testing should therefore not be limited to successful transactions. It must also verify how the system behaves when unexpected conditions occur.

Procedure / testing process

1. Requirement analysis

Understand the complete fund-transfer process, including account validation, amount validation, balance checking, transaction confirmation and failure handling.

2. Functional testing

Test normal transfers using valid accounts and valid amounts.

3. Negative testing

Test invalid account numbers, invalid amounts, insufficient balance and expired sessions.

4. Boundary testing

Test minimum and maximum permitted transfer amounts.

5. Network testing

Test the application when the network is:

- Disconnected
- Slow
- Interrupted during transaction
- Restored after failure

6. Concurrency testing

Test multiple simultaneous transactions to identify race conditions and duplicate processing.

7. Failure analysis

Use crash logs, application logs, transaction records and reproduction steps to identify the root cause.

8. Retesting and regression

After fixing the crash, repeat the failed test and execute related transaction tests.

Practical application / example

Suppose a user transfers ₹10,000.

The system debits the account, but the network disconnects before the confirmation reaches the application.

The tester should verify:

1. Whether the transaction is recorded.
2. Whether the amount is deducted only once.
3. Whether the transaction is marked pending if required.
4. Whether the user can safely retry.
5. Whether duplicate deduction occurs.

If the application crashes and the amount is deducted twice after retrying, the issue can be classified as a serious transaction-handling defect.

Conclusion

Systematic functional, negative, boundary, network and concurrency testing can identify the conditions responsible for mobile banking crashes. Proper regression testing and failure handling reduce future transaction failures and improve application reliability.

7.

Aim

To differentiate testing from debugging and establish an effective process for integrating both activities into software development.

Introduction

Testing and debugging are related but different activities.

Testing is the process of evaluating software to identify defects or failures.

Debugging is the process of investigating the cause of a defect and modifying the software to correct it.

Testing generally answers:

"is something wrong?"

Debugging investigates:

"why is it wrong, and how can it be fixed?"

Procedure / testing process

1. Test the software

The tester executes the software using planned test cases.

2. Identify failure

If actual behavior differs from expected behavior, the failure is documented.

3. Report defect

The tester provides steps to reproduce the issue, expected result and actual result.

4. Perform debugging

The developer investigates code, logs, database operations and program flow to identify the root cause.

5. Apply fix

The developer corrects the underlying defect.

6. Retest

The tester executes the original failed test case again.

7. Regression testing

Related functionality is tested to ensure the fix has not caused new defects.

Practical application / example

Suppose an atm deducts ₹5,000 when the customer requested ₹2,000.

The tester executes the withdrawal and identifies the incorrect deduction. This is testing.

The developer examines the withdrawal calculation and discovers that the amount variable is incorrectly processed. Correcting the program is debugging.

The tester then executes the withdrawal again to verify the fix.

Conclusion

Testing identifies failures and provides evidence of defects, while debugging identifies and removes their causes. The process:

Testing → defect report → debugging → fix → retesting → regression

Ensures effective defect resolution and improves software quality.

8.

Aim

To design a systematic cross-browser testing strategy for an e-commerce application and ensure consistent behavior across supported browsers and devices.

Introduction

Web applications may behave differently across browsers because browsers use different rendering engines, javascript implementations, css support and device environments.

An e-commerce application must therefore be tested across the browsers and devices supported by its users.

Important functions include login, product search, product display, cart operations, checkout and payment.

Procedure / testing process

1. Identify supported environments

Prepare a browser matrix containing supported browsers, operating systems, devices and screen sizes.

2. Identify critical functions

Prioritize important workflows such as:

- Login
- Search
- Product selection
- Add to cart
- Checkout
- Payment

3. Execute functional tests

Execute the same test cases across different browsers.

4. Test ui compatibility

Verify fonts, buttons, menus, images, forms and layouts.

5. Test responsive behavior

Check different screen sizes and mobile devices.

6. Test javascript and css behavior

Verify that interactive components behave correctly across supported browsers.

7. Report browser-specific defects

Record the browser, version, operating system, steps, expected result and actual result.

8. Retest and regression test

After fixing compatibility defects, repeat the affected tests across supported environments.

Practical application / example

Suppose the checkout button works correctly in chrome but cannot be clicked in another supported browser.

The tester compares the same workflow across browsers and discovers that a css or javascript compatibility issue prevents the button from functioning.

The defect is reported with the browser and version information so that developers can reproduce it.

Conclusion

Cross-browser testing identifies compatibility problems before users experience them. A systematic browser matrix, functional testing, responsive testing and regression testing improve reliability, compatibility and user experience.

9.

Aim

To classify logical defects in a payroll system and apply suitable testing techniques to prevent incorrect salary calculations.

Introduction

Payroll systems perform important calculations such as basic salary, allowances, deductions, taxes and net salary. Incorrect logic can result in wrong salary amounts.

A defect can be classified according to its nature. Incorrect salary calculations generally represent functional or logical defects, while incorrect employee information may represent a data defect.

Procedure / testing process

1. Analyze requirements

Understand salary calculation rules and business conditions.

2. Identify calculation logic

For example:

Net salary = basic salary + allowances – deductions

3. Identify defect classes

Possible problems include:

- Functional defects
- Calculation defects
- Data defects
- Performance defects
- Interface defects

4. Design test cases

Test different salary ranges, employee types, deductions and allowances.

5. Boundary testing

Test values at tax or salary limits.

6. Decision table testing

Test combinations such as employee type, tax status and applicable allowances.

7. Regression testing

After modifying salary logic, verify previously working calculations.

Practical application / example

Suppose an employee has:

- Basic salary = ₹30,000
- Allowance = ₹5,000
- Deduction = ₹2,000

The expected net salary is:

$$₹30,000 + ₹5,000 - ₹2,000 = ₹33,000$$

If the program calculates ₹37,000 because it adds the deduction instead of subtracting it, the calculation logic contains a defect.

The defect can be reported, corrected and added to the regression suite.

Conclusion

Defect classification helps identify the nature of payroll problems and select suitable testing techniques. Boundary, decision-table, calculation and regression testing can prevent logical errors from reaching production.

10. A software team skips proper test case design and directly executes random tests. Apply testing as an engineering activity to redesign the testing process. Explain the importance of structured test case design. Illustrate how this improves defect detection. Suggest suitable techniques to enhance test effectiveness.

Aim

To replace random testing with systematic test case design and improve the effectiveness and coverage of software testing.

Introduction

Ad-hoc testing involves testing software without a formally designed set of test conditions. Although it may sometimes discover unexpected defects, it does not guarantee adequate coverage.

Structured test case design derives tests from requirements and uses defined testing techniques.

Procedure / testing process

1. Study requirements

Identify what the system should do.

2. Identify test conditions

Determine valid, invalid, boundary and exceptional conditions.

3. Select test design techniques

Use:

- Equivalence partitioning
- Boundary value analysis
- Decision table testing
- State transition testing
- Use case testing

4. Prepare test cases

Each test case should contain:

- Test case id
- Preconditions
- Input
- Steps
- Expected result

5. Execute tests

Compare actual results with expected results.

6. Report defects

Document failures clearly.

7. Retest and regression test

Verify fixes and ensure existing functionality remains correct.

Practical application / example

For an online payment system, random testing might involve trying a normal payment amount.

Structured testing would additionally test:

- Valid card

- Invalid card
- Expired card
- Zero amount
- Negative amount
- Maximum amount
- Network failure
- Duplicate payment request

This provides significantly broader coverage.

Conclusion

Structured test case design makes testing systematic, repeatable, traceable and measurable. It improves defect detection and reduces the risk of important conditions being missed.

11.

Aim

To analyze data inconsistency across healthcare application modules and establish a systematic approach to ensure data integrity.

Introduction

Data consistency means that the same information remains accurate and synchronized wherever it is used.

A healthcare application may contain modules for patient registration, doctor consultation, pharmacy, laboratory reports and billing. If one module contains updated information while another contains old information, data inconsistency occurs.

Such defects may originate from incorrect database updates, integration problems, synchronization failures, incorrect data mapping or transaction failures.

Procedure / testing process

1. Identify data requirements

Determine which data should be shared between modules.

2. Test data creation

Verify that newly created patient data is correctly stored.

3. Test data modification

Change patient information and verify that all relevant modules receive the update.

4. Test integration

Verify communication between modules.

5. Test database consistency

Compare application values with database values.

6. Test concurrent updates

Check what happens when multiple modules update the same information.

7. Test failure recovery

Interrupt transactions and verify rollback or recovery behavior.

8. Report defects

Document the affected modules and expected and actual data.

Practical application / example

Suppose a doctor updates a patient's medication information.

The doctor module displays the new medication, but the pharmacy module still displays the old medication.

The tester records:

Expected: pharmacy should display the updated medication.

Actual: pharmacy displays the previous medication.

This may indicate an integration or synchronization defect.

Conclusion

Systematic database, integration and data-validation testing improves data integrity across healthcare modules. Proper defect tracking and regression testing help prevent repeated inconsistencies.

12.

Aim:

To use defect repository information to identify repeated defect patterns and improve future test design and defect prevention.

Introduction:

A company may perform testing regularly and still experience repeated defects if historical defect information is not analyzed.

A defect repository stores information about defects from different releases. When this information is analyzed, testers can identify patterns such as frequently affected modules, common root causes and repeated defect types.

Procedure / testing process:

1. Record defects

Every discovered defect is entered into the repository.

2. Classify defects

Defects are categorized according to module, type, severity and priority.

3. Record root cause

The team records whether the defect resulted from requirements, design, coding, data or integration.

4. Analyze historical data

Compare defects across releases.

5. Identify patterns

Look for modules or functions where defects repeatedly occur.

6. Improve test design

Create new test cases specifically for recurring problem areas.

7. Add regression tests

Every important recurring defect should become part of the regression suite.

8. Monitor future releases

Continue analyzing new defect data.

Practical application / example

Suppose a banking application has the following payment defects:

- Release 1: 12
- Release 2: 15
- Release 3: 14

Repository analysis shows that many defects are related to payment calculations.

The team can introduce additional calculation tests, boundary tests and automated regression tests.

Conclusion:

A defect repository is not simply a storage system. When analyzed effectively, it becomes a source of historical knowledge that improves test planning, identifies recurring patterns and prevents repeated defects.

13.

Aim:

To identify performance-testing gaps in a web-based learning system and improve reliability during peak usage.

Introduction:

A web application may work correctly with a small number of users but fail when thousands of users access it simultaneously.

This indicates that functional testing alone was insufficient. Performance testing is required to determine whether the system can handle expected and unexpected workloads.

Important performance testing types include load testing, stress testing, volume testing and concurrency testing.

Procedure / testing process:

1. Identify expected load

Determine the expected number of simultaneous users.

2. Define performance requirements

Establish acceptable response times, throughput and error rates.

3. Design load scenarios

Simulate realistic user behavior such as:

- Login
- Opening courses
- Downloading materials
- Submitting assignments
- Taking examinations

4. Perform load testing

Gradually increase the number of users.

5. Perform stress testing

Increase the load beyond expected capacity to determine system limits.

6. Monitor resources

Observe cpu, memory, database and network utilization.

7. Identify bottlenecks

Analyze slow responses, server overload and database problems.

8. Retest after optimization

Repeat performance tests after improvements.

Practical application / example

Suppose a college learning portal normally handles 500 users but an online examination may bring 5,000 students online simultaneously.

The tester can simulate:

500 → 1,000 → 2,500 → 5,000 users

The tester measures response time, throughput, error rate and server resource usage.

If the application begins failing at 3,000 users, the team has identified a capacity problem before the actual examination.

Conclusion:

Proper performance testing identifies system limitations before deployment. Load, stress and concurrency testing improve reliability and ensure that the learning platform can handle peak usage.

14.

Aim:

To involve testers during the early stages of sdlc and improve software quality through early defect identification.

Introduction:

Traditionally, testing was sometimes treated as an activity performed after development. Modern testing practices encourage testers to participate from the requirements and design stages.

This approach is called early testing or shift-left testing.

Defects detected during requirements or design are generally easier to correct than defects discovered after deployment because late defects may require significant rework.

Procedure / testing process:

1. Requirement phase

Testers review requirements and identify ambiguity, missing conditions and contradictions.

2. Design phase

Testers examine whether the design supports the required functionality and whether it can be effectively tested.

3. Development phase

Testers prepare test cases while developers implement the software.

4. Integration phase

Testers verify communication between components.

5. System testing

The complete application is tested against requirements.

6. Acceptance testing

The application is checked against user and business requirements.

7. Maintenance

Defects found after deployment are analyzed and added to regression testing where appropriate.

Practical application / example

Suppose a banking requirement says:

"money should be transferred instantly."

The word "instantly" is ambiguous.

A tester involved during requirements analysis can ask:

What is the maximum acceptable transaction response time?

The team can then define a measurable requirement such as a specific response-time target.

This prevents ambiguity from becoming a development and testing problem later.

Conclusion:

Early tester involvement improves requirements, identifies risks sooner, reduces rework and improves collaboration between developers, testers and business teams. It ultimately improves software quality while reducing the cost of fixing defects.

15.

Aim:

To identify gaps in test coverage and improve test case design so that important system conditions are properly validated.

Introduction

Test coverage represents how much of the required functionality, conditions or scenarios have been tested.

A system may produce correct results for the cases that were tested while still failing for cases that were never tested. Therefore, executing a large number of test cases does not necessarily mean that coverage is sufficient.

Procedure / testing process:

1. Analyze requirements

List all functional and non-functional requirements.

2. Identify input classes

Use equivalence partitioning to identify valid and invalid input groups.

3. Identify boundaries

Use boundary value analysis for minimum and maximum limits.

4. Identify condition combinations

Use decision tables when multiple conditions affect the result.

5. Identify system states

Use state transition testing when behavior changes according to system state.

6. Map requirements to tests

A requirement traceability matrix can be used to ensure every requirement has corresponding test cases.

7. Identify coverage gaps

Find requirements or scenarios that have no test cases.

8. Add missing tests

Create additional positive, negative, boundary and failure-condition tests.

Practical application / example

Suppose an atm allows withdrawal between ₹100 and ₹50,000.

Testing only ₹5,000 does not provide sufficient coverage.

The tester should test:

- ₹99
- ₹100
- ₹101
- ₹49,999
- ₹50,000

- ₹50,001
- Insufficient balance
- Invalid pin
- Invalid card
- Network failure

This ensures that the system is tested beyond normal conditions.

Conclusion:

Proper test coverage reduces the probability of missing important defects. Using systematic test design techniques and traceability provides better validation and improves system reliability.

16.

Aim:

To improve defect reporting in a logistics application by using clear definitions, structured reports and standardized reporting practices.

Introduction:

A defect report is a formal record describing an observed problem in software. Poorly written reports may cause developers to misunderstand the issue or spend excessive time trying to reproduce it.

A good defect report should clearly communicate what happened, where it happened, how it can be reproduced and what should have happened.

Procedure / testing process:

1. Identify the failure

Observe where the actual result differs from the expected result.

2. Reproduce the problem

Repeat the steps to confirm that the issue is reproducible.

3. Record environment

Mention browser, operating system, application version or other relevant environment information.

4. Write reproduction steps

Steps should be clear and sequential.

5. Record expected and actual results

This allows the developer to immediately understand the difference.

6. Assign severity and priority

Severity represents the impact of the defect, while priority represents how urgently it should be addressed.

7. Attach evidence

Screenshots, logs and videos can support the report when appropriate.

8. Retest the fix

After correction, the tester verifies the original issue.

Practical application / example

Poor report:

"delivery time is wrong."

Better report:

Title: delivery time does not update after changing destination.

Steps:

1. Open an existing order.
2. Change the delivery destination.
3. Select the delivery option.
4. Confirm the order.

Expected: delivery time should be recalculated according to the new destination.

Actual: the previous delivery time remains displayed.

This report allows the developer to reproduce the issue easily.

Conclusion:

Structured defect reporting improves communication, reproducibility and resolution speed. Clear defect descriptions also improve tracking and help organizations identify recurring problems.

17. A software team struggles to maintain consistency in testing across modules. Apply process-based testing approaches to solve the issue. Explain how standardized processes improve quality. Illustrate your approach with an example. Suggest methods to ensure consistency.

Aim:

To establish a standardized process-based testing approach so that testing remains consistent across different software modules.

Introduction:

When different teams test software using different methods, important scenarios may be missed and defect reports may not be consistent.

Process-based testing establishes a common sequence of activities, documentation formats, standards and quality criteria.

A common testing process is:

Requirement analysis → test planning → test design → environment setup → test execution → defect reporting → retesting → regression testing → closure

Procedure / testing process:

1. Standardize requirement analysis

All testers should review requirements using a common approach.

2. Standardize test planning

Testing scope, resources, risks and schedules should be documented.

3. Standardize test cases

A common template should be used for test case id, input, steps and expected result.

4. Standardize defect reports

All teams should use common defect fields and severity definitions.

5. Establish entry and exit criteria

Testing should begin and end based on defined conditions.

6. Perform reviews

Test cases and defect reports should be reviewed before execution or closure.

7. Perform regression testing

Common regression suites should be maintained for important functionality.

Practical application / example

Suppose three banking teams test:

- Login
- Fund transfer
- Payment

One team classifies a major transaction failure as "high," while another classifies a similar failure as "critical."

A common severity standard ensures that similar defects are classified consistently.

Similarly, using the same test case template allows managers to compare results across modules.

Conclusion:

Process-based testing creates consistency, repeatability and traceability across modules. Standardized test cases, defect reports, criteria and regression practices improve the overall quality of the testing process.

18.

Aim:

To identify why edge cases are missed and apply boundary value analysis to prevent failures after software deployment.

Introduction:

Edge cases are conditions that occur at the extreme limits of acceptable input. They are important because programming errors frequently occur at boundaries.

Boundary value analysis (bva) focuses on testing values at, below and above the boundaries of an input range.

Procedure / testing process

1. Identify the input range

Determine the minimum and maximum permitted values.

2. Identify boundary values

Find the lower and upper limits.

3. Test values below the boundary

Check invalid values immediately outside the permitted range.

4. Test boundary values

Check the exact minimum and maximum values.

5. Test values above the boundary

Check values immediately above the permitted range.

6. Combine with other conditions

Boundary values should also be tested with relevant conditions such as insufficient balance or invalid authentication.

7. Report and correct defects

Any unexpected behavior should be reported and corrected.

8. Regression test

Repeat boundary tests after changes.

Practical application / example

Suppose an atm permits withdrawals from ₹100 to ₹50,000.

The boundary tests are:

Input	Expected
₹99	Rejected
₹100	Accepted
₹101	Accepted
₹49,999	Accepted
₹50,000	Accepted
₹50,001	Rejected

Suppose the programmer accidentally writes:

Amount < 50000

Instead of:

Amount <= 50000

Then ₹50,000 will be incorrectly rejected. Boundary testing detects this defect.

Conclusion:

Boundary value analysis ensures that edge conditions are explicitly tested. It is especially useful for financial systems, registration limits, age limits, file sizes and transaction amounts. Including boundary cases reduces production failures caused by incorrect limit handling.

19. A company lacks proper documentation of defects, making it difficult to track issues. Apply defect repository concepts to address this problem. Explain how documentation improves defect management. Illustrate how structured repositories support analysis. Suggest improvements in documentation practices.

Aim:

To establish a structured defect repository and improve defect documentation, tracking, analysis and prevention.

Introduction:

Defect documentation is an essential part of software testing because defects need to be tracked from discovery until resolution.

A defect repository provides a centralized location for maintaining this information. Without proper documentation, teams may lose track of unresolved issues, repeat the same tests unnecessarily or fail to identify recurring defect patterns.

Procedure / testing process

1. Identify the defect

The tester observes unexpected behavior.

2. Assign a unique id

Each defect receives a unique identification number.

3. Record complete information

The report should contain:

- Title
- Module
- Environment
- Preconditions
- Steps to reproduce
- Expected result
- Actual result
- Severity
- Priority
- Evidence

4. Assign the defect

The issue is assigned to the appropriate developer or team.

5. Record resolution

The repository should contain information about the correction and root cause.

6. Retest

The tester verifies the correction.

7. Close the defect

The issue is closed only after successful verification.

8. Analyze historical data

Teams should periodically examine the repository for recurring modules, defect types and root causes.

Practical application / example

Suppose login defects repeatedly occur whenever the password-change feature is modified.

The defect repository may reveal:

- Several defects from the same module.
- Similar root causes.
- Repeated regression failures.

The testing team can respond by creating a dedicated password-management regression suite and increasing testing whenever the authentication module is modified.

Conclusion:

Proper defect documentation provides traceability from discovery to closure. Repository analysis provides historical knowledge that can improve test planning and prevent repeated defects, resulting in better software quality.

20.

Aim:

To redesign the testing approach of a banking transaction system using software testing principles and systematic techniques to reduce repeated transaction failures.

Introduction:

Banking transaction systems are highly dependent on accuracy, consistency and reliability. Transactions such as fund transfers, withdrawals and payments must handle both normal and abnormal conditions correctly.

Repeated failures can occur because of insufficient test coverage, random test selection, inadequate negative testing, poor regression testing or failure to consider boundary and network conditions.

Therefore, a structured and risk-based testing approach is required.

Procedure / testing process:

1. Requirement analysis

Identify all transaction requirements, including:

- Account validation
- Transaction amount
- Balance requirements
- Transaction limits
- Authentication
- Transaction status
- Failure handling
- Rollback requirements

2. Risk-based test planning

High-risk operations such as fund transfer and payment should receive greater testing attention because failures may have significant consequences.

3. Test case design

Use systematic techniques.

Equivalence		partitioning:
divide transaction inputs into valid and invalid categories.		

Boundary	value	analysis:
test transaction limits.		

Decision		tables:
test combinations such as valid account + sufficient balance + valid amount.		

State	transition	testing:
test transaction states such as:		

Initiated → processing → successful / failed / pending

4. Positive testing

Verify that valid transactions are processed successfully.

5. Negative testing

Test invalid accounts, insufficient balances, invalid amounts and authentication failures.

6. Failure and recovery testing

Test network failures, server failures and interrupted transactions.

7. Defect reporting

Record every failure with complete reproduction information.

8. Retesting

Verify that corrected defects are actually resolved.

9. Regression testing

Execute related transaction tests after every significant change.

10. Automation

Automate repetitive transaction and regression scenarios to improve consistency.

Practical application / example

Consider a fund transfer of ₹10,000.

A particularly important scenario is network interruption.

Suppose the account is debited but the confirmation message is lost due to a network failure. The tester must verify that retrying the transaction does not cause a second debit.

This type of systematic failure testing can identify defects that normal successful-transaction testing would miss.

Conclusion:

A systematic banking testing process combines requirement analysis, risk-based planning, structured test design, positive and negative testing, boundary testing, failure testing, defect tracking, retesting, regression testing and automation.

This approach provides broader coverage and improves the detection of transaction-related defects. As a result, repeated failures, incorrect deductions and duplicate transactions can be reduced, improving the reliability and overall quality of the banking system.

Code of Conduct:

I B S Reshma (192521342) (Name / Reg No) certify that this submission is my original work and

that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

B S Reshma

Rubrics for calculation:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand